

Class

类

类（class）是结构体的拓展，不仅能够拥有成员元素，还拥有成员函数。

在面向对象编程（OOP）中，对象就是类的实例，也就是变量。

C++ 中 `struct` 关键字定义的也是类，上文中的 **结构体** 的定义来自 C。因为某些历史原因，C++ 保留并拓展了 `struct`。

定义类

类使用关键字 `class` 或者 `struct` 定义，下文以 `class` 举例。

```
1 class ClassName {
2     ...
3 };
4
5 // Example:
6 class Object {
7 public:
8     int weight;
9     int value;
10 } e[array_length];
11
12 const Object a;
13 Object b, B[array_length];
14 Object *c;
```

与使用 `struct` 大同小异。该例定义了一个名为 `Object` 的类。该类拥有两个成员元素，分别为 `weight,value`；并在 `}` 后使用该类型定义了一个数组 `e`。

定义类的指针形同 [struct](#)。

访问说明符

不同于 [struct](#) 中的举例，本例中出现了 `public`，这属于访问说明符。

- `public`：该访问说明符之后的各个成员都可以被公开访问，简单来说就是无论 **类内** 还是 **类外** 都可以访问。
- `protected`：该访问说明符之后的各个成员可以被 **类内**、派生类或者友元的成员访问，但类外 **不能访问**。
- `private`：该访问说明符之后的各个成员 **只能被类内** 成员或者友元的成员访问，**不能** 被从类外或者派生类中访问。

对于 `struct`，它的所有成员都是默认 `public`。对于 `class`，它的所有成员都是默认 `private`。

关于 "友元" 和 "派生类"，可以参考下方折叠框，或者查询网络资料进行详细了解。

对于算法竞赛来说，友元和派生类并不是必须要掌握的知识点。

► 关于友元以及派生类的基本概念

访问与修改成员元素的值

方法形同 [struct](#)

- 对于变量，使用 . 符号。
- 对于指针，使用 -> 符号。

成员函数

成员函数，顾名思义。就是类中所包含的函数。

► 常见成员函数举例

```
1 class Class_Name {
2     ... type Function_Name(...) { ... }
3 };
4
5 // Example:
6 class Object {
7     public:
8     int weight;
9     int value;
10
11     void print() {
12         cout << weight << endl;
13         return;
14     }
15
16     void change_w(int);
17 };
18
19 void Object::change_w(int _weight) { weight = _weight; }
20
21 Object var;
```

该类有一个打印 `Object` 成员元素的函数，以及更改成员元素 `weight` 的函数。

和函数类似，对于成员函数，也可以先声明，在定义，如第十四行（声明处）以及十七行后（定义处）。

如果想要调用 `var` 的 `print` 成员函数，可以使用 `var.print()` 进行调用。

重载运算符

► 何为重载

重载运算符，可以部分程度上代替函数，简化代码。

下面给出重载运算符的例子。

```

1 class Vector {
2 public:
3     int x, y;
4
5     Vector() : x(0), y(0) {}
6
7     Vector(int _x, int _y) : x(_x), y(_y) {}
8
9     int operator*(const Vector& other) const { return x * other.x + y * other.y; }
10
11     Vector operator+(const Vector&) const;
12     Vector operator-(const Vector&) const;
13 };
14
15 Vector Vector::operator+(const Vector& other) const {
16     return Vector(x + other.x, y + other.y);
17 }
18
19 Vector Vector::operator-(const Vector& other) const {
20     return Vector(x - other.x, y - other.y);
21 }
22
23 // 关于4,5行表示为x,y赋值，具体实现参见后文。

```

该例定义了一个向量类，并重载了 $*$ $+$ $-$ 运算符，并分别代表向量内积，向量加，向量减。

重载运算符的模板大致可分为下面几部分。

```

1 /*类定义内重载*/ 返回类型 operator符号(参数){...}
2
3 /*类定义内声明，在外部定义*/ 返回类型 类名称::operator符号(参数){...}

```

对于自定义的类，如果重载了某些运算符（一般来说只需要重载 $<$ 这个比较运算符），便可以使用相应的 STL 容器或算法，如 [sort](#)。

如要了解更多，可参见「参考资料」第四条。

► 可以被重载的运算符

在实例化变量时设定初始值

为完成这种操作，需要定义 **默认构造函数(Default constructor)**。

```

1 class ClassName {
2     ... ClassName(...)... { ... }
3 };
4
5 // Example:
6 class Object {
7 public:
8     int weight;
9     int value;
10
11     Object() {
12         weight = 0;
13         value = 0;
14     }
15 };

```

该例定义了 `Object` 的默认构造函数，该函数能够在我们实例化 `Object` 类型变量时，将所有的成员元素初始化为 0。

若无显式的构造函数，则编译器认为该类有隐式的默认构造函数。换言之，若无定义任何构造函数，则编译器会自动生成一个默认构造函数，并会根据成员元素的类型进行初始化（与定义 内置类型 变量相同）。

在这种情况下，成员元素都是未初始化的，访问未初始化的变量的结果是未定义的（也就是说并不知道会返回何值）。

如果需要自定义初始化的值，可以再定义（或重载）构造函数。

► 关于定义（或重载）构造函数

使用 C++11 或以上时，可以使用 {} 进行变量的初始化。

► 关于 {}

```
1 class Object {
2 public:
3     int weight;
4     int value;
5
6     Object() {
7         weight = 0;
8         value = 0;
9     }
10
11     Object(int _weight = 0, int _value = 0) {
12         weight = _weight;
13         value = _value;
14     }
15
16     // the same as
17     // Object(int _weight,int _value):weight(_weight),value(_value) {}
18 };
19
20 // the same as
21 // Object::Object(int _weight,int _value){
22 //     weight = _weight;
23 //     value = _value;
24 // }
25 //}
26
27 Object A;           // ok
28 Object B(1, 2);     // ok
29 Object C{1, 2};     // ok, (C++11)
```

► 关于隐式类型转换

销毁

这是不可避免的问题。每一个变量都将在作用范围结束走向销毁。

但对于已经指向了动态申请的内存的指针来说，该指针在销毁时不会自动释放所指向的内存，需要手动释放动态内存。

如果结构体的成员元素包含指针，同样会遇到这种问题。需要用到析构函数来手动释放动态内存。

析构 函数（Destructor）将会在该变量被销毁时被调用。重载的方法形同构造函数，但需要在前加 ~

默认定义的析构函数通常对于算法竞赛已经足够使用，通常我们只有在成员元素包含指针时才会重载析构函数。

```
1 class Object {
2 public:
3     int weight;
4     int value;
5     int* ned;
6
7     Object() {
8         weight = 0;
9         value = 0;
10    }
11
12    ~Object() { delete ned; }
13};
```

为类变量赋值

默认情况下，赋值时会按照对应成员元素赋值的规则进行。也可以使用 `类名称()` 或 `类名称{}` 作为临时变量来进行赋值。

前者只是调用了复制构造函数（copy constructor），而后者在调用复制构造函数前会调用默认构造函数。

另外默认情况下，进行的赋值都是对应元素间进行 **浅拷贝**，如果成员元素中有指针，则在赋值完成后，两个变量的成员指针具有相同的地址。

```
1 // A,tmp1,tmp2,tmp3类型为Object
2 tmp1 = A;
3 tmp2 = Object(...);
4 tmp3 = {...};
```

如需解决指针问题或更多操作，需要重载相应的构造函数。

更多 **构造函数 (constructor)** 内容，参见「参考资料」第六条。

参考资料

1. [cppreference class](#)
2. [cppreference access](#)
3. [cppreference default_constructor](#)
4. [cppreference operator](#)
5. [cplusplus Data structures](#)
6. [cplusplus Special members](#)
7. [C++11 FAQ](#)
8. [cppreference Friendship and inheritance](#)
9. [cppreference value initialization](#)

本页面最近更新：2024/10/9 22:38:42, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [lr1d](#), [JasonkayZK](#), [Lans1ot](#), [CCXXxi](#), [Chrogeek](#), [cjsoft](#), [Enter-tainer](#), [kernel_panic](#), [ksyx](#), [Lewy Zeng](#), [Tiphereth-A](#), [YiJiugg](#)

本页面的全部内容 [在 CC BY-SA 4.0 和 SATA 协议之条款下](#) 提供，附加条款亦可能应用